

Smart Library Circulation & Automation System (SLCAS)

COS 202 — Project Report

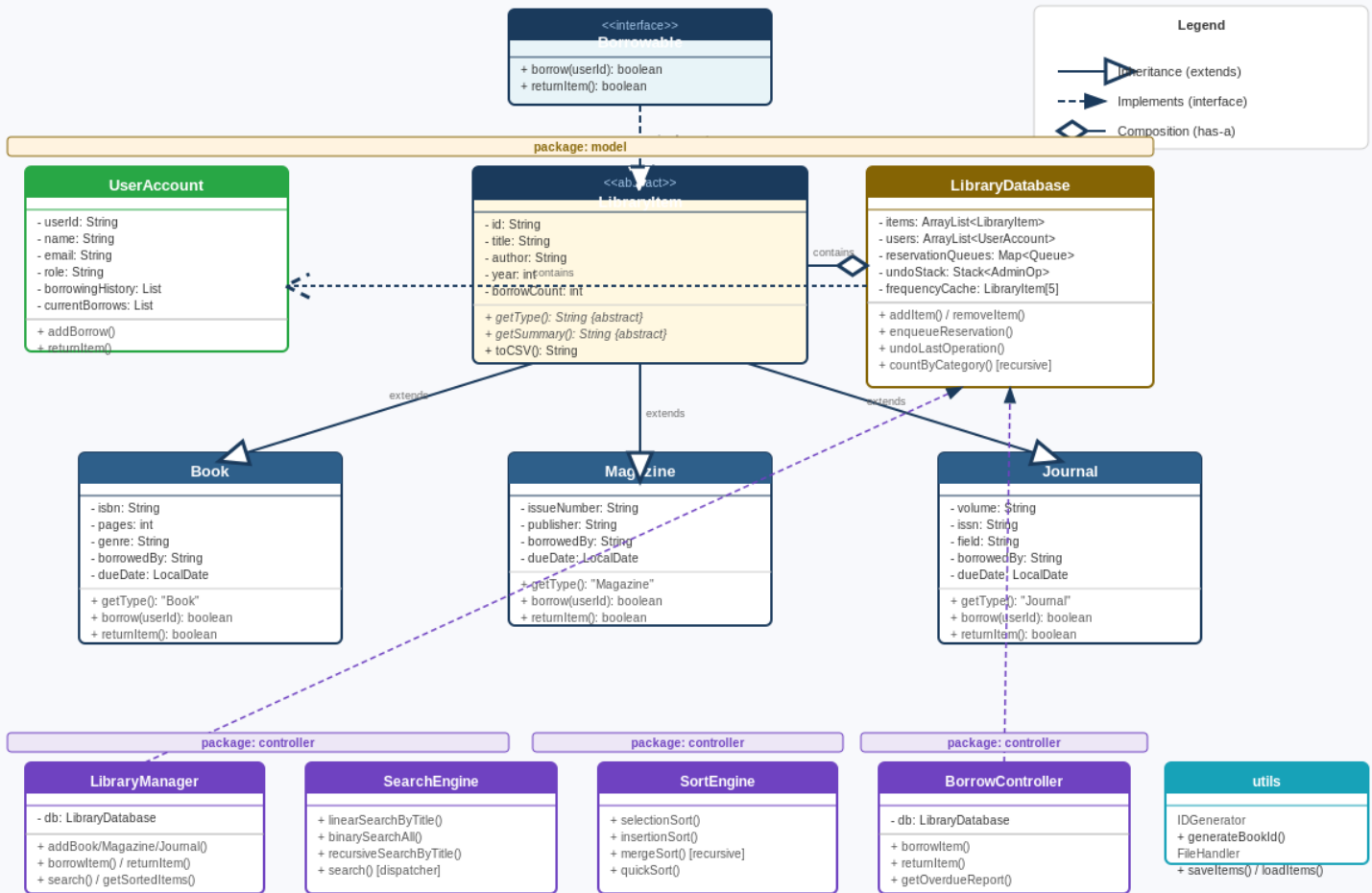
Course: COS 202

University: MIVA Open University

Date: Sat Mar 07 2026

SLCAS — UML Class Diagram

COS 202 | Smart Library Circulation & Automation System



1. System Description

The Smart Library Circulation & Automation System (SLCAS) is a fully functional desktop application built in Java that manages a university library's books, magazines, and journals. The system provides a comprehensive graphical user interface (GUI) for library staff and administrators to manage library items, handle borrowing and returning workflows, maintain reservation queues, and generate reports.

The application was designed with a strong emphasis on object-oriented programming principles, efficient data structures, and well-known sorting and searching algorithms. It demonstrates the practical use of abstract classes, interfaces, polymorphism, encapsulation, and composition in a real-world scenario.

SLCAS supports full data persistence through CSV file storage, ensuring that all library data is saved and restored across application sessions. The GUI was built using Java Swing with event-driven programming, tabbed panels, dialog boxes, and dynamic components.

2. Features Implemented

2.1 Core Features

- Add new library items: Books, Magazines, and Journals with full metadata
- Borrow and return items with automatic due date calculation
- Reservation queue: users are added to a waitlist when an item is unavailable
- Search items by title, author, or type using selectable algorithms
- Sort items by title, author, or year using selectable algorithms
- Undo last admin operation (add or delete) using a Stack
- GUI dashboard with 5 tabbed panels
- Save and load all data from CSV files automatically
- Generate reports: most borrowed items, overdue users, category distribution
- Overdue reminder timer that checks for overdue items periodically

2.2 GUI Panels

- View Items — table view of all items with type filter and custom status renderer
- Borrow/Return — forms for borrowing and returning, reservation queue display
- Admin — add items (Book/Magazine/Journal), delete items, manage users, undo
- Search & Sort — live search with algorithm picker, sort with algorithm picker
- Reports — most borrowed table, overdue report, category bar chart

3. Data Structures Used

The system makes use of four distinct data structures as required by the project specification, each chosen for a specific purpose:

Structure	Class Used	Location	Purpose
ArrayList	java.util.ArrayList	LibraryDatabase	Store all library items and user accounts
Queue (LinkedList)	java.util.LinkedList	LibraryDatabase	Per-item reservation/waitlist for borrowed books
Stack	java.util.Stack	LibraryDatabase	Undo stack for last admin add/delete operation
Array (fixed size)	LibraryItem[]	LibraryDatabase	Fixed-size cache of top 5 most frequently borrowed items

The ArrayList provides $O(1)$ average-case access and dynamic resizing for the main item collection. The Queue (FIFO) is ideal for the reservation system, ensuring users are served in the order they requested. The Stack (LIFO) enables natural undo functionality, reversing the most recent operation first. The fixed-size Array cache provides $O(1)$ access to frequently accessed items, demonstrating the trade-off between speed and flexibility.

4. Algorithms Chosen and Why

4.1 Search Algorithms

Three search algorithms were implemented to demonstrate different approaches and trade-offs:

- **Linear Search:** Linear Search — scans every element in order. Used when the list is unsorted or when searching by author or type. Time complexity: $O(n)$. Chosen for its simplicity and universal applicability regardless of sort state.
- **Binary Search:** Binary Search — divides the sorted list in half repeatedly. Used when searching by title and the list is sorted. Time complexity: $O(\log n)$. Chosen for its efficiency on large sorted datasets.
- **Recursive Search:** Recursive Search — traverses the list recursively from index 0 to n . Demonstrates the recursive programming paradigm. Used as an alternative to linear search. Time complexity: $O(n)$.

4.2 Sorting Algorithms

Four sorting algorithms were implemented, all selectable from a GUI dropdown:

- **Selection Sort:** Selection Sort — finds the minimum element and swaps it into position. Simple but $O(n^2)$ in all cases. Suitable for small datasets.
- **Insertion Sort:** Insertion Sort — builds the sorted array one element at a time. $O(n^2)$ worst case but $O(n)$ for nearly-sorted data. Efficient for small or nearly sorted lists.

- **Merge Sort (default):** Merge Sort — recursive divide-and-conquer. Guaranteed $O(n \log n)$ in all cases. Chosen as the default algorithm for its predictable performance and stability.
- **Quick Sort:** Quick Sort — partitions around a pivot element. Average $O(n \log n)$, worst $O(n^2)$. Chosen for its in-place sorting and real-world speed advantages.

4.3 Recursive Algorithms

- `countByCategory()` — recursively counts items matching a category by traversing the list from index to `n`
- `computeOverdueCharge()` — recursively computes the total fine across all overdue borrow records
- `recursiveSearchByTitle()` / `recursiveSearchByAuthor()` — recursive list traversal for searching
- `mergeSort()` — recursive divide-and-conquer sorting algorithm

5. Challenges Faced

- **GUI Layout Complexity** — arranging multiple panels with `BorderLayout`, `GridBagLayout`, and `CardLayout` required careful planning to avoid components overlapping or misaligning. The Admin panel's `CardLayout` for switching between Book/Magazine/Journal forms was particularly tricky to implement cleanly.
- **Data Persistence** — designing a CSV format that could represent three different item types (Book, Magazine, Journal) with different fields required a flexible parsing strategy. Handling commas within field values (e.g., titles) required substituting semicolons during serialization.
- **Reservation Queue Integration** — linking the Queue to the borrow/return workflow required careful synchronization. When an item is returned, the next user in the queue must be identified and notified via the GUI status bar.
- **Binary Search Edge Cases** — the binary search implementation needed to handle prefix matching (not just exact matches) and expand left and right from the found index to collect all matching results.
- **Undo Stack Consistency** — ensuring the undo stack correctly restored deleted items to the `ArrayList` and removed re-added items required careful management of object references.
- **Cross-panel Refresh** — changes made in the Admin or Borrow panels needed to immediately reflect across all other panels (e.g., View Items table). This was solved by calling `refreshAllTabs()` after every data-modifying operation.

6. Object-Oriented Design Summary

OOP Concept	Applied In	How
Abstraction	LibraryItem (abstract)	Defines common interface for all item types
Interface	Borrowable	Enforces borrow/return contract on items
Inheritance	Book, Magazine, Journal	Extend LibraryItem, inherit common fields
Polymorphism	getSummary(), getType()	Each subclass returns its own type/summary
Encapsulation	All model classes	Private fields with getters/setters
Composition	LibraryDatabase	Contains ArrayList of items, users, queues, stack

The project is organized into four packages: model (data classes), controller (business logic), gui (Swing interface), and utils (helper utilities). This separation of concerns follows the Model-View-Controller (MVC) pattern, making the codebase maintainable and extensible.